ENGG1110 Problem Solving by Programming (25-26 Term 1)

Final Project

Deadline: **28-Nov**-2025 (Friday) 23:59

---

1. Introduction

A maze [1] is a complex network of paths or passages designed as a puzzle through which one must find a way from a starting point to an exit. Mazes often include dead ends, multiple routes, and sometimes hidden shortcuts or traps, making navigation a challenge.

```
Welcome to the Maze Game!
Use 8(up) 2(down) 4(left) 6(right) to move or -1 to quit the game.
Reach 'EX' to win.
Maze:
[][][][][][][][][][]
[]@@  mr[]        []
[]  []  []  [][]  []
[]  []        []  []
[]  [][]EX[]  []  []
[]        []  []  []
[][][][][]  []  []  []
[]    []  [][][]  []
[]                []
[][][][][][][][][]EX[]
```

Maze in this project

In this project, the game board is a fixed 10×10 rectangular grid where each cell is either a wall, a walkable space, or a special item. There may be multiple exits in this project. Assume the given maze is solvable, and the border cells of the game board can only be either walls or exit.
The player navigates the maze using directional input (8 for up, 2 for down, 4 for left, 6 for right, -1 for quit), aiming to reach the exit cell. Along the way, the player may encounter two special items:

- Mirror: flip the maze horizontally. (in Part one)
- Rotation: rotates the maze 90° clockwise. (in Part two)

The game ends when the player successfully reaches the exit, or input –1 to quit the game.

[1] Wikipedia contributors, "Maze," Wikipedia, The Free *Encyclopedia*, https://en.wikipedia.org/wiki/Maze

2. Schedule

The recommended project schedule is as follows (new test cases will be released at different checkpoints):

Week	Date	Part One Tasks (Final Submission Date: 28 Nov)
9	Oct 27-31	• Complete the following functions: ○ **`printMaze()`** • In `main()`: ○ Get user input direction and check validation • Test your code in Gradescope Project Part 1
10	Nov 3-7	• Complete the following functions: ○ **`movePlayerOneStep()`** • In `main()`: ○ Complete the implementation of the game process ○ Check reach the exit or not • Re-test your code in Gradescope Project Part 1
11	Nov 10-14	• Complete the following function: ○ **`mirrorMaze()`** • In `main()`: ○ Call **`mirrorMaze()`** • Design your own test cases and perform the final testing of Project Part 1 • Re-test your code in Gradescope Project Part 1
Week	**Lab Date**	**Part Two Tasks** **(Final Submission Date: 28 Nov)**
12	Nov 17-21	• Complete the following functions: ○ **`initGameBoardFromFile()`** ○ **`rotateMaze90()`** • Update the function **`printMaze()`** • In `main()`: ○ Test `initGameBoardFromFile()` and `printMaze()` • Test your code in Gradescope Project Part 2
13	Nov 24-28	• Complete the following function: ○ **`movePlayerMultiSteps()`** • Design your own test cases and perform the final testing of Project Part 2 • Re-test your code in Gradescope Project Part 2

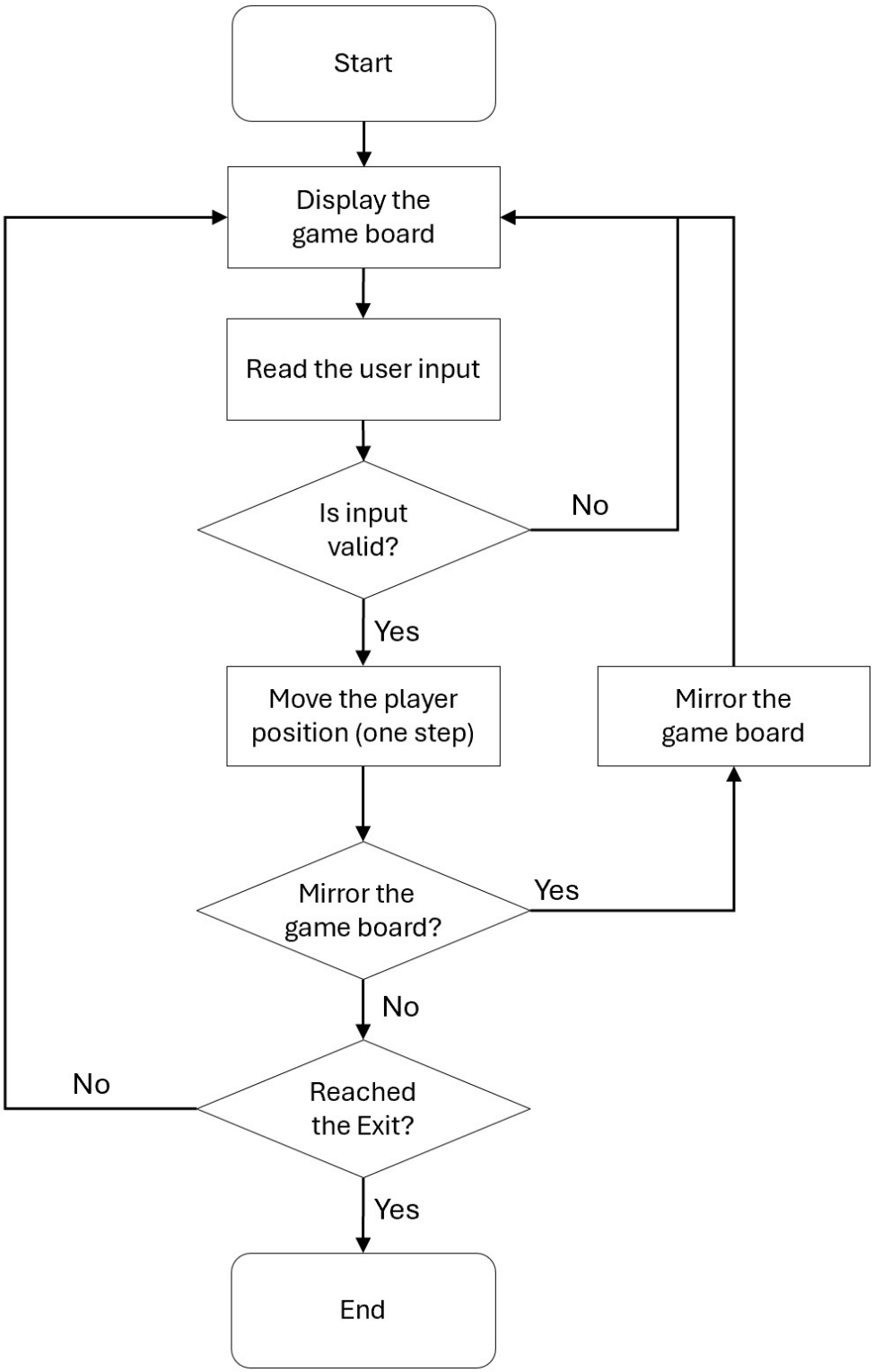
DO NOT change the given functions, including the return type and parameters provided in the given file "Project.c" Any changes will result in a mark deduction.

Note: passing all test cases **DOES NOT** guarantee getting full marks because there will be other hidden test cases and grading processes after the project final submission date!

3. Project Part 1

- In Project Part 1, your task is to implement the game of Maze that user can only move one step at a time.
- You are required to complete the given **Project.c** without modifying the existing code. Please rename it to **ProjectPart1_yourStudentID.c** like `ProjectPart1_1155123456.c` .

The program flow in Part 1 is shown as follows:



4. Detailed Program Design (Part 1)

4.1 Header Files, Variable Declarations and Macros

Apart from <stdio.h> and <stdlib.h>, do NOT use definitions/functions/macros from other sources. NO global variables (those declared outside functions) are allowed. That is, all variables must be declared inside functions, thus local. Information must be exchanged through parameter passing and return value.

The following macros are defined in the beginning of the given source code:

```
/* Macros (symbolic constants) */
#define WIDTH 10
#define HEIGHT 10
#define ROW 0
#define COL 1
```

These symbolic constants can also reduce the chance of typos. Such macros are usually in all CAPS and defined at the beginning of a source file.

In addition, the following local variables is declared with these initializations in the main function:

```
char mazeGameBoard [HEIGHT][WIDTH] = {
      {'W','W','W','W','W','W','W','W','W','W'},
      {'W','.','.','M','W','.','.','.','.','W'},
      {'W','.','W','.','W','.','W','W','.','W'},
      {'W','.','W','.','.','.','.','W','.','W'},
      {'W','.','W','W','E','W','.','W','.','W'},
      {'W','.','.','.','.','W','.','W','.','W'},
      {'W','W','W','W','.','W','.','W','.','W'},
      {'W','.','.','W','.','W','W','W','.','W'},
      {'W','.','.','.','.','.','.','.','.','W'},
      {'W','W','W','W','W','W','W','W','E','W'}
    };
int player[2], steps = 0;
player[ROW]=1;
player[COL]=1;
```

The 2-D array **mazeGameBoard** stores the layout of the maze, where each cell may contain one of the following elements:

- **'W'** – Wall (impassable)
- **'.'** – Empty walkable space (walkable)
- **'M'** – Mirror item (mirrors the maze horizontally)
- **'E'** – Exit (goal of the game)

The given maze is solvable, and the border cells of the game board are either walls (**'W'**) or exits (**'E'**).

The integer array **player** stores the player's current position. In the code, use **player[ROW]** and **player[COL]** to refer to the player's row and column position respectively, which are set to row 1 and column 1 in Part One.

The integer **steps** is for keeping track of the number of steps taken. It is initialized to 0 at the beginning.

4.2 Print Maze Game Board

The **`main()`** function invokes the **`printMaze()`** function, which displays the maze game board on the screen.

For each cell in **`mazeGameBoard`**, if it matched:

- The player's current coordinates (array **`player`**), it prints **`"@@"`** to represent the player's current position,
- **`'W'`**, it prints **`"[]"`** to represent a wall.
- **`'E'`**, it prints **`"EX"`** (uppercase) to indicate the exit.
- **`'M'`**, it prints **`"mr"`** (lowercase) to represent a mirror item.
- **`'.'`**, it prints two spaces **`"  "`** for empty walkable space.

According to the initialization of **`mazeGameBoard`**, the initial maze layout should appear as follows . Note that an additional line, **`"Maze:"`** , is printed before the game board.

```
Maze:
[][][][][][][][][][]
[]@@  mr[]        []
[]  []  []  [][]  []
[]  []        []  []
[]  [][]EX[]  []  []
[]        []  []  []
[][][][]  []  []  []
[]    []  [][][]  []
[]                []
[][][][][][][][]EX[]
```

4.3 Print Current Steps and Get User Input

In each round of the game, the **`main()`** function displays the maze game board, prints current steps, and prompts the player to enter the move direction.

The program prints the following message about the steps after the display of game board and before asking for input in the **`main()`** function, which records how many grid cells the player has moved through. Using the initial output, the **`step`** is 0 as an example:

```
Steps: 0
```

Then, the program prints the following prompts in each round of game:

```
Enter move direction (8/2/4/6) or -1 to quit the game:
```

and then read in an integer **input** representing the movement direction,

- 8 – Move up
- 2 – Move down
- 4 – Move left
- 6 – Move right.
- -1 – Quit the game.

We define the **input** integer and direction according to the Numeric keypad:



If the **input** is -1, the following message will be displayed, and the program will terminate immediately:

```
You chose to quit the maze.
=====Game Over=====
```

If the **input** is invalid (Assume the input must be an integer, but may be not one of the accepted values), the program prints the following error message and prompts the player to try again:

```
Invalid input!
Please use 8(up) 2(down) 4(left) 6(right) or -1(quit).
```

4.4 Move one step

After receiving user input, the **main()** function invokes the **movePlayerOneStep()** function to update the player's position. This function returns the number of grid cells moved, which is then used to update the local variable **steps** in **main().**

In the **movePlayerOneStep()** function, the input direction is first mapped to changes in row and column coordinates. The function then checks whether the intended move is valid:

- If the move is invalid, the following message is printed:

  ```
  You hit a wall!
  ```

  The function then returns 0 to **main()** without updating the player's position.

- If the move is valid, the player's position (array **player**) is updated accordingly, and returns 1 to the **main()** function.

4.5 Win the maze game

In the **main()** function, after each successful move, the game checks whether the player has reached the exit. If the player's current position matches the exit cell **'E'**, the following congratulatory message is displayed with step number, X is the current value of **steps**.

```
You reached the exit in X steps.
=====Congratulations=====
```

4.6 Mirror the maze game board

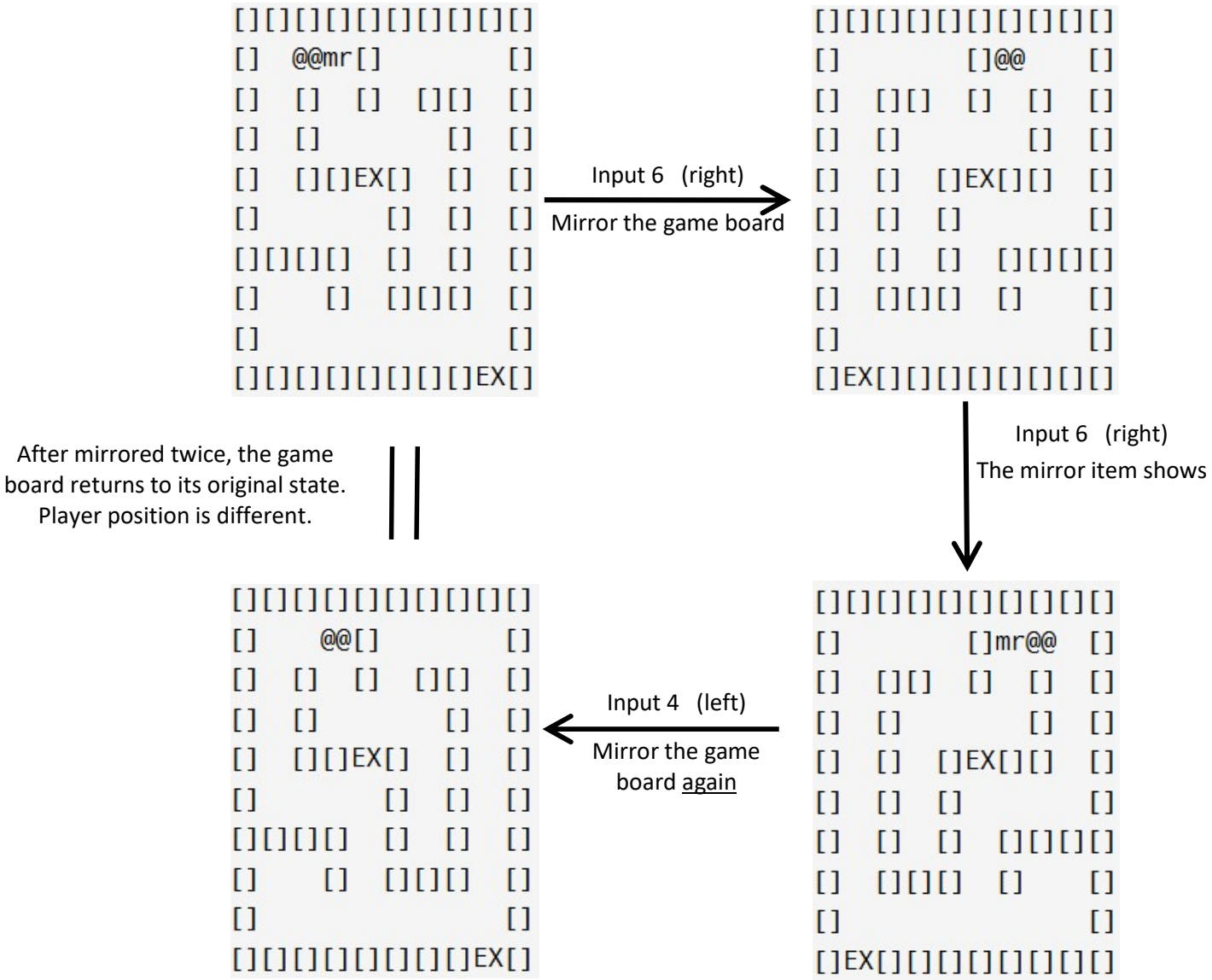
In the **`main()`** function, after each successful move, the game checks whether the player has stepped onto a mirror item **`'M'`**.

If so, the following message is displayed:

```
You found a mirror item! Maze is now mirrored.
```

Then, the maze is horizontally mirrored by calling the **`mirrorMaze()`** function. This transformation swaps the left and right halves of the maze, updates the player's position accordingly, and the mirror item becomes hidden behind the player.

Here is one example:



4.7. Sample Run (Part 1)

One sample run is shown below (**underlined bold characters in blue** are user inputs). In addition to it, you are strongly recommended to test your program by designing your own test cases.

```
Welcome to the Maze Game!
Use 8(up) 2(down) 4(left) 6(right) to move or -1 to quit
the game.
Reach 'EX' to win.
Maze:
[][][][][][][][][][]
[]@@  mr[]        []
[]  []  []  [][]  []
[]  []        []  []
[]  [][]EX[]  []  []
[]        []  []  []
[][][][]  []  []  []
[]    []  [][][]  []
[]                []
[][][][][][][][][]EX[]
Steps: 0
Enter move direction (8/2/4/6) or -1 to quit the game: 9
Invalid input!
Please use 8(up) 2(down) 4(left) 6(right) or -1(quit).
Maze:
[][][][][][][][][][]
[]@@  mr[]        []
[]  []  []  [][]  []
[]  []        []  []
[]  [][]EX[]  []  []
[]        []  []  []
[][][][]  []  []  []
[]    []  [][][]  []
[]                []
[][][][][][][][][]EX[]
Steps: 0
Enter move direction (8/2/4/6) or -1 to quit the game: 8
You hit a wall!
Maze:
[][][][][][][][][][]
[]@@  mr[]        []
[]  []  []  [][]  []
[]  []        []  []
[]  [][]EX[]  []  []
[]        []  []  []
[][][][]  []  []  []
[]    []  [][][]  []
[]                []
[][][][][][][][][]EX[]
Steps: 0
Enter move direction (8/2/4/6) or -1 to quit the game: 6
Maze:
[][][][][][][][][][]
[]  @@mr[]        []
[]  []  []  [][]  []
[]  []        []  []
[]  [][]EX[]  []  []
[]        []  []  []
[][][][]  []  []  []
[]    []  [][][]  []
[]                []
[][][][][][][][][]EX[]
Steps: 1
Enter move direction (8/2/4/6) or -1 to quit the game: 4
Maze:
[][][][][][][][][][]
[]@@  mr[]        []
[]  []  []  [][]  []
[]  []        []  []
[]  [][]EX[]  []  []
[]        []  []  []
[][][][]  []  []  []
[]    []  [][][]  []
[]                []
[][][][][][][][][]EX[]
Steps: 2
Enter move direction (8/2/4/6) or -1 to quit the game: 6
Maze:
[][][][][][][][][][]
[]  @@mr[]        []
[]  []  []  [][]  []
[]  []        []  []
[]  [][]EX[]  []  []
[]        []  []  []
[][][][]  []  []  []
[]    []  [][][]  []
[]                []
[][][][][][][][][]EX[]
Steps: 3
Enter move direction (8/2/4/6) or -1 to quit the game: 6
You found a mirror item! Maze is now mirrored.
Maze:
[][][][][][][][][][]
[]        []@@    []
[]  [][]  []  []  []
[]  []        []  []
[]  []  []EX[][]  []
[]  []  []        []
[]  []  []  [][][][]
[]  [][][]  []    []
[]                []
[]EX[][][][][][][][]
Steps: 4
Enter move direction (8/2/4/6) or -1 to quit the game: 2
Maze:
[][][][][][][][][][]
[]        []mr    []
[]  [][]  []@@[]  []
[]  []        []  []
[]  []  []EX[][]  []
[]  []  []        []
[]  []  []  [][][][]
[]  [][][]  []    []
[]                []
[]EX[][][][][][][][]
Steps: 5
Enter move direction (8/2/4/6) or -1 to quit the game: 2
Maze:
[][][][][][][][][][]
[]        []mr    []
[]  [][]  []  []  []
[]  []      @@[]  []
[]  []  []EX[][]  []
[]  []  []        []
[]  []  []  [][][][]
[]  [][][]  []    []
[]                []
[]EX[][][][][][][][]
Steps: 6
Enter move direction (8/2/4/6) or -1 to quit the game: 4
Maze:
[][][][][][][][][][]
[]        []mr    []
[]  [][]  []  []  []
[]  []    @@  []  []
[]  []  []EX[][]  []
[]  []  []        []
[]  []  []  [][][][]
[]  [][][]  []    []
[]                []
[]EX[][][][][][][][]
Steps: 7
Enter move direction (8/2/4/6) or -1 to quit the game: 2
You reached the exit in 8 steps.
=====Congratulations=====
```

5. Project Part 2

In Project Part 2, your task is to implement the game of Maze that user can **move multiple steps at a time**.

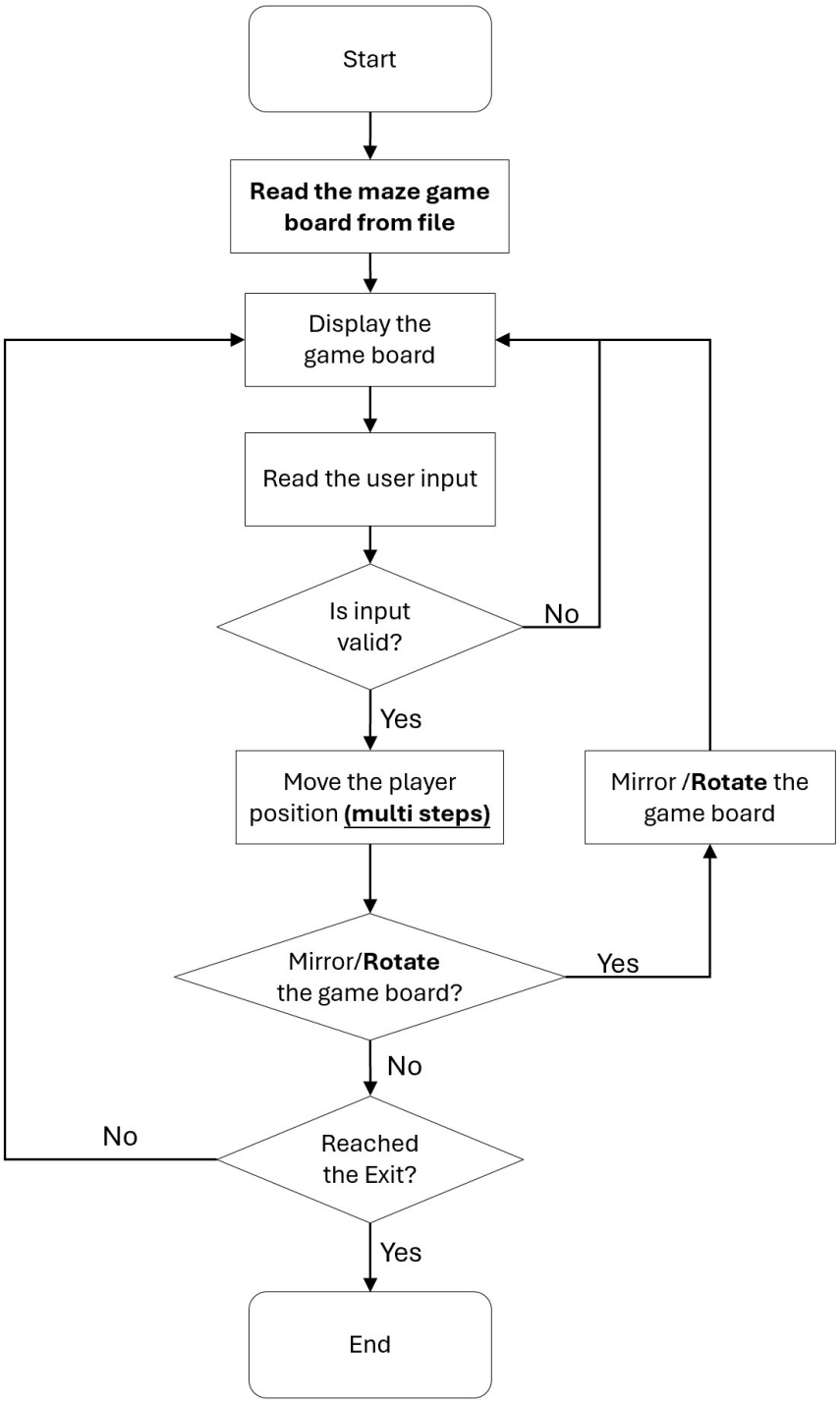
You are required to:

a) copy your completed **ProjectPart1_yourStudentID.c**;

b) rename it to **ProjectPart2_yourStudentID.c** like `ProjectPart2_1155123456.c`

c) add the extra functionalities, test, debug, finish and submit the work.

The program flow in Part 2 is shown as follows:



6. Detailed Program Design (Part 2)

6.1 Header Files, Variable Declarations and Macros

Governing rules on header files, macros remain and local variables declaration the same as Part 1. However, you need to initialize the game board and the player's position from a file with one new special item rotation **'R'** is included in part two. Please refer to Section 6.2 for more details.

6.2 Initialize a Game Board from a file

The **main()** function invokes the **initGameBoardFromFile()** function, which initializes the **mazeGameBoard** array and set player starting position (array **player**).

Similar to Part one, each cell in game board can only contain one of the following elements:

- **'W'** – Wall (impassable)
- **'.'** – Empty walkable space (walkable)
- **'R'** – Rotation item (rotates the maze 90° clockwise), *new in Part Two*.
- **'M'** – Mirror item (mirrors the maze horizontally)
- **'E'** – Exit (goal of the game)

The file "gameboard.txt" contains a 10*10 characters representing the maze layout, followed by two integers indicating the player's starting position.

It is assumed that:

- The file format is correct.
- The maze consists only of the characters **'W'**, **'.'**, **'E'**, **'M'**, and **'R'**.
- The maze must be a valid configuration, fully bordered except for the exits.
- The border cells of the game board can only be either walls (**'W'**) or exits (**'E'**).
- The player's starting position is in a walkable space.

A sample "gameboard.txt" is shown below:

gameboard.txt	Display
WWWWWWWWWW W...M....W W.WWWW.W.W W......W.W W.WWWR.W.E W......W.W W.WRWWE..W W.W..W.WWW W........W WWWWWWWWEWW 3 1	[]][][][][][][][][][] [] mr [] [] [][][][] [] [] []@@ [] [] [] [][][]rc [] EX [] [] [] [] []rc[][]EX [] [] [] [] [][][] [] [] [][][][][][][][]EX[][]

If fails to open the "gameboard.txt" file, the following message will be displayed and terminate the program immediately:

```
Error: Unable to open gameboard.txt.
=====Game Over=====
```

6.3 Print Game Board

Similar to Part 1, The **`main()`** function invokes the **`printMaze()`** function and prints the full game board with one additional item, roration. That is for each cell in **`mazeGameBoard`**, if it matched:

- The player's position, it prints **`"@@"`** .
- **`'W'`**, it prints **`"[]"`** to represent a wall.
- **`'E'`**, it prints **`"EX"`** (uppercase) to indicate the exit.
- **`'R'`**, it prints **`"rc"`** (lowercase) to represent a rotation item, *new in Part Two*.
- **`'M'`**, it prints **`"mr"`** (lowercase) to represent a mirror item.
- **`'.'`**, it prints two spaces **`"  "`** for empty walkable space.

6.4 Print Current Steps and Get User Input

Same as Part 1, in each round of the game, the **`main()`** function displays the maze game board, prints the current steps and prompts the player to enter the move direction.

6.5 Move Multiple Steps

After receiving user input, the **`main()`** function invokes the **`movePlayerMultiSteps()`** function instead of `movePlayerOneStep()` function, to attempt moving the player *multiple steps* in the input direction. This function returns the number of grid cells moved, which is then used to update the local variable **`steps`** in `main()` function.

Similar to Part 1 `movePlayerOneStep()`, the **`movePlayerMultiSteps()`** function first translates the input direction into changes in row and column coordinates. It then checks whether the initial move is valid:

- If the initial move is invalid, the following messages is printed:

  ```
  You hit a wall!
  ```

  The function then returns 0 to **`main()`** without updating the player's position.

- If the initial move is valid, the player continues moving in the chosen direction, updating the player position (array **`player`**) until one of the following conditions is met:
  - ✧ The player encounters a wall in the next cell.
  - ✧ The player encounters a special item in the next cell, that is either a mirror (**`'M'`**) or a rotation item (**`'R'`**).
  - ✧ The player lands directly on **`'E'`** Exit, **`'M'`** Mirror or **`'R'`** Rotation.
  - ✧ The player reaches a point where multiple walkable directions are available (**`'E'`** Exit, **`'M'`** Mirror and **`'R'`** Rotation are also walkable).

  And then return the number of grid cells moved to the **`main()`** function.

  Here is one long example showing different cases:

Long Example for Multi Steps

The player encounters a wall in the next cell.

The player encounters a special item in the next cell.

The player lands directly on 'M' Mirror.

The player reaches a point where multiple walkable directions are available.

Long Example for Multi Steps

The player encounters a wall in the next cell.

The player reaches a point where multiple walkable directions are available.

```
Maze:
[][][][][][][][][][]
[]      mr        []
[]  [][][][]  []  []
[]@@          []  []
[]  [][][]rc  []  EX
[]            []  []
[]  []rc[][]EX    []
[]  []    []  [][][]
[]                []
[][][][][][][][]EX[][]
Steps: 0
Enter move direction (8/2/4/6) or -1 to quit the game: 8
Maze:
[][][][][][][][][][]
[]@@    mr        []
[]  [][][][]  []  []
[]            []  []
[]  [][][]rc  []  EX
[]            []  []
[]  []rc[][]EX    []
[]  []    []  [][][]
[]                []
[][][][][][][][]EX[][]
Steps: 2
Enter move direction (8/2/4/6) or -1 to quit the game: 6
Maze:
[][][][][][][][][][]
[]    @@mr        []
[]  [][][][]  []  []
[]            []  []
[]  [][][]rc  []  EX
[]            []  []
[]  []rc[][]EX    []
[]  []    []  [][][]
[]                []
[][][][][][][][]EX[][]
Steps: 4
Enter move direction (8/2/4/6) or -1 to quit the game: 6
You found a mirror item! Maze is now mirrored.
Maze:
[][][][][][][][][][]
[]        @@      []
[]  []  [][][][]  []
[]  []            []
EX  []  rc[][][]  []
[]  []            []
[]    EX[][]rc[]  []
[][][]  []    []  []
[]                []
[][]EX[][][][][][][]
Steps: 5
Enter move direction (8/2/4/6) or -1 to quit the game: 4
Maze:
[][][][][][][][][][]
[]    @@  mr      []
[]  []  [][][][]  []
[]  []            []
EX  []  rc[][][]  []
[]  []            []
[]    EX[][]rc[]  []
[][][]  []    []  []
[]                []
[][]EX[][][][][][][]
Steps: 7
Enter move direction (8/2/4/6) or -1 to quit the game: 4
Maze:
[][][][][][][][][][]
[]@@      mr      []
[]  []  [][][][]  []
[]  []            []
EX  []  rc[][][]  []
[]  []            []
[]    EX[][]rc[]  []
[][][]  []    []  []
[]                []
[][]EX[][][][][][][]
Steps: 9
Enter move direction (8/2/4/6) or -1 to quit the game: 2
Maze:
[][][][][][][][][][]
[]        mr      []
[]  []  [][][][]  []
[]  []            []
EX@@[]  rc[][][]  []
[]  []            []
[]    EX[][]rc[]  []
[][][]  []    []  []
[]                []
[][]EX[][][][][][][]
Steps: 12
Enter move direction (8/2/4/6) or -1 to quit the game: 4
You reached the exit in 13 steps.
=====Congratulations=====
```

6.6 Win the maze game

Same as Part 1, if the player has reached the exit, end the game with number of steps moved printed.

6.7 Mirror the maze game board

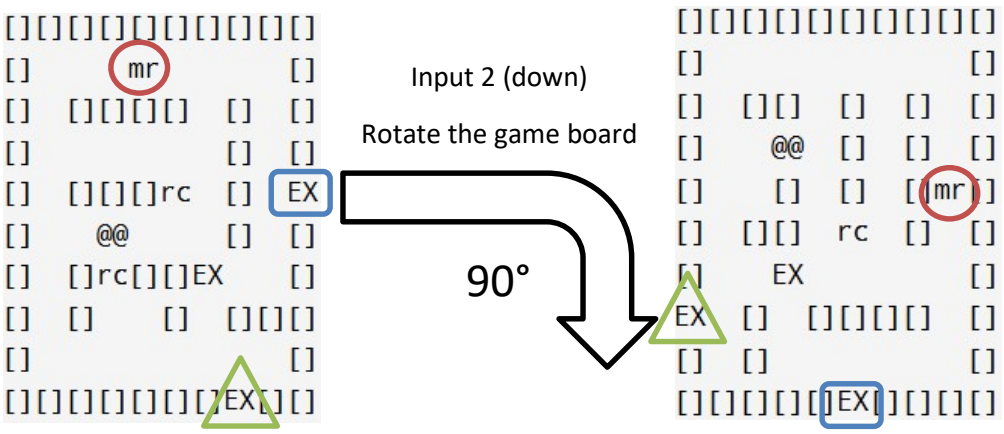
Same as Part 1, if the player has stepped onto a mirror item, mirror the maze game board and player position.

6.8 Rotate the maze game board

In the **`main()`** function, after each successful move, the game checks whether the player has stepped onto a rotation item **`'R'`**. If so, the following message is displayed:

```
You found a rotation item! Maze is now rotated.
```

The maze is then rotated 90 degrees clockwise by calling the **`rotateMaze90()`** function. This transformation rotates both the maze layout and the player's position accordingly. After rotation, the rotation item is hidden beneath the player's new position but still stored in the maze.



Hint: The relationship between rotate, transpose and mirror.

6.8. Sample Run (Part 2)

One sample run is shown below (**underlined bold characters in blue** are user **inputs**). In addition to it, you are strongly recommended to test your program by designing your own test case.

```
Welcome to the Maze Game!
Use 8(up) 2(down) 4(left) 6(right) to move or -1 to quit
the game.
Reach 'EX' to win.
Maze:
[][][][][][][][][][]
[]      mr        []
[]  [][][][]  []  []
[]@@          []  []
[]  [][][]rc  []  EX
[]            []  []
[]  []rc[][]EX    []
[]  []    []  [][][]
[]                []
[][][][][][][][]EX[][]
Steps: 0
Enter move direction (8/2/4/6) or -1 to quit the game: 6
Maze:
[][][][][][][][][][]
[]      mr        []
[]  [][][][]  []  []
[]        @@  []  []
[]  [][][]rc  []  EX
[]            []  []
[]  []rc[][]EX    []
[]  []    []  [][][]
[]                []
[][][][][][][][]EX[][]
Steps: 4
Enter move direction (8/2/4/6) or -1 to quit the game: 2
You found a rotation item! Maze is now rotated.
Maze:
[][][][][][][][][][]
[]                []
[]  [][]  []  []  []
[]    rc  []  []  []
[]    []  []  []mr[]
[]  [][]  @@  []  []
[]    EX          []
EX  []  [][][][]  []
[]  []            []
[][][][][][EX][][][]
Steps: 5
Enter move direction (8/2/4/6) or -1 to quit the game: 4
Maze:
[][][][][][][][][][]
[]                []
[]  [][]  []  []  []
[]    rc  []  []  []
[]    []  []  []mr[]
[]  [][]@@rc  []  []
[]    EX          []
EX  []  [][][][]  []
[]  []            []
[][][][][][EX][][][]
Steps: 6
Enter move direction (8/2/4/6) or -1 to quit the game: 8
Maze:
[][][][][][][][][][]
[]                []
[]  [][]  []  []  []
[]    rc@@[]  []  []
[]    []  []  []mr[]
[]  [][]  rc  []  []
[]    EX          []
EX  []  [][][][]  []
[]  []            []
[][][][][][EX][][][]
Steps: 8
Enter move direction (8/2/4/6) or -1 to quit the game: 4
```

```
You found a rotation item! Maze is now rotated.
Maze:
[][]EX[][][][][][][]
[]                []
[][][]  []    []  []
[]    EX[][]@@[]  []
[]  []            []
EX  []  rc[][][]  []
[]  []            []
[]  []  [][][][]  []
[]        mr      []
[][][][][][][][][][]
Steps: 9
Enter move direction (8/2/4/6) or -1 to quit the game: 8
Maze:
[][]EX[][][][][][][]
[]                []
[][][]  []  @@[]  []
[]    EX[][]rc[]  []
[]  []            []
EX  []  rc[][][]  []
[]  []            []
[]  []  [][][][]  []
[]        mr      []
[][][][][][][][][][]
Steps: 10
Enter move direction (8/2/4/6) or -1 to quit the game: 8
Maze:
[][]EX[][][][][][][]
[]          @@    []
[][][]  []    []  []
[]    EX[][]rc[]  []
[]  []            []
EX  []  rc[][][]  []
[]  []            []
[]  []  [][][][]  []
[]        mr      []
[][][][][][][][][][]
Steps: 11
Enter move direction (8/2/4/6) or -1 to quit the game: 4
Maze:
[][]EX[][][][][][][]
[]        @@      []
[][][]  []    []  []
[]    EX[][]rc[]  []
[]  []            []
EX  []  rc[][][]  []
[]  []            []
[]  []  [][][][]  []
[]        mr      []
[][][][][][][][][][]
Steps: 12
Enter move direction (8/2/4/6) or -1 to quit the game: 4
Maze:
[][]EX[][][][][][][]
[]    @@          []
[][][]  []    []  []
[]    EX[][]rc[]  []
[]  []            []
EX  []  rc[][][]  []
[]  []            []
[]  []  [][][][]  []
[]        mr      []
[][][][][][][][][][]
Steps: 14
Enter move direction (8/2/4/6) or -1 to quit the game: 2
You reached the exit in 16 steps.
=====Congratulations=====
```

Note: The player moved to the exit directly in this case.

7. Academic Honesty and Declaration Statement

Attention is drawn to University policy and regulations on honesty in academic work, and to the disciplinary guidelines and procedures applicable to breaches of such policy and regulations. Details may be found at

http://www.cuhk.edu.hk/policy/academichonesty (CUHK)

https://www.erg.cuhk.edu.hk/erg/AcademicHonesty (Engineering Faculty)

Please place the following declaration statement as the comment in the beginning of your two .c source code files and fill in your information. Submissions without the properly signed declaration will not be graded.

Tools such as software similarity measurement, AI-tool, fingerprint detection, etc., might be used to inspect all submissions.

```
/**
 * ENGG1110 Problem Solving by Programming
 *
 * Course Project
 *
 * I declare that the project here submitted is original
 * except for source material explicitly acknowledged,
 * and that the same or closely related material has not been
 * previously submitted for another course.
 * I also acknowledge that I am aware of University policy and
 * regulations on honesty in academic work, and of the disciplinary
 * guidelines and procedures applicable to breaches of such
 * policy and regulations, as contained in the website.
 *
 * I have read and understood the University's policy on
 * the use of AI for academic work. I confirm that I have complied
 * with the instructions given by my course teacher(s) regarding the
 * use of AI tools for this project work and consent to the
 * use of AI content detection software to review my submission.
 *
 * University Guideline on Academic Honesty:
 *   https://www.cuhk.edu.hk/policy/academichonesty/
 *
 * Student Name  : <your name>
 * Student ID    : <your student ID>
 * Class/Section : <your class/section>
 * Date          : <date>
 */
```

8. Testing Platform

You are recommended to work in **CLion** on Windows or macOS.
However, any IDE is acceptable — as long as your program produces the **same output** as the Gradescope.

9. Submission

Please follow the steps below to submit your work by the deadline specified on the first page.

1. Your work may be graded with the aid of Gradescope autograder, under markers' supervision and further inspection. **Please note that while passing the test cases is necessary, it is *not sufficient* on its own for getting full marks.** Additional marks will be allocated based on the completeness and adherence to all requirements specified in the project documentation, including code organization, functionality, and adherence to the design specifications.
2. Submit **ProjectPart1_yourStudentID.c** like `ProjectPart1_1155123456.c` and **ProjectPart2_yourStudentID.c** like `ProjectPart2_1155123456.c` to the Gradescope.
    - Note: No need to submit other files used in your IDE (if any)

The late submission penalty is as follows:

- Within 3 days (72 hours): 10% per 24 hours
- More than 3 days but within 7 days: 50% penalty
- More than 7 days: 100% penalty (you will receive 0 marks in the project AND fail the course)

NOTE: Finishing and submitting the project properly is crucial in PASSING the course.